Allocators from C to Zig

An allocator is a tool that reserves memory (typically on the heap) so a program can store its data structures there. Many C programs use the standard libc allocator, or at best, let you switch it out for another one like jemalloc or mimalloc.

Unlike C, modern systems languages usually treat allocators as first-class citizens. Let's look at how they handle allocation and then create a C allocator following their approach.

Rust

Rust is one of the older languages we'll be looking at, and it handles memory allocation in a more traditional way. Right now, it uses a global allocator, but there's an experimental Allocator API implemented behind a feature flag (issue #32838). We'll set the experimental API aside and focus on the stable one.

Global allocator

The documentation begins with a clear statement:

> In a given program, the standard library has one "global" memory allocator that is used for example by `Box<T>` and `Vec<T>`.

Followed by a vague one:

> Currently the default global allocator is unspecified.

It doesn't mean that a Rust program will abort an allocation, of course. In practice, Rust uses the system allocator as the global default (but the Rust developers don't want to commit to this, hence the "unspecified" note):

- `malloc` on Unix platforms;
- `HeapAlloc` on Windows;
- `dlmalloc` in WASM.

The global allocator interface is defined by the `GlobalAlloc` trait in the `std::alloc` module. It requires the implementor to provide two essential methods — `alloc` and `dealloc`, and provides two more based on them — `alloc_zeroed` and `realloc`:

```rust
pub unsafe trait GlobalAlloc {
    // Allocates memory as described by the given `layout`.
    // Returns a pointer to newly-allocated memory,
    // or null to indicate allocation failure.
    unsafe fn alloc(&self, layout: Layout) -> *mut u8;

    // Deallocates the block of memory at the given `ptr`
    // pointer with the given `layout`.
    unsafe fn dealloc(&self, ptr: *mut u8, layout: Layout);

    // Behaves like `alloc`, but also ensures that the contents
    // are set to zero before being returned.
    unsafe fn alloc_zeroed(&self, layout: Layout) -> *mut u8 {
        // ...
    }

    // Shrinks or grows a block of memory to the given `new_size` in bytes.
    // The block is described by the given `ptr` pointer and `layout`.
    unsafe fn realloc(&self, ptr: *mut u8, layout: Layout, new_size: usize) -> *mut u8 {
        // ...
    }
}
```

Layout

The `Layout` struct describes a piece of memory we want to allocate — its size in bytes and alignment:

```rust
pub struct Layout {
    // private fields
    size: usize,
    align: Alignment,
}
```

Memory alignment

Alignment restricts where a piece of data can start in memory. The memory address for the data has to be a multiple of a certain number, which is always a power of 2.

Alignment depends on the type of data:

- `u8`: alignment = 1. Can start at any address (0, 1, 2, 3...).
- `i32`: alignment = 4. Must start at addresses divisible by 4 (0, 4, 8, 12...).
- `f64`: alignment = 8. Must start at addresses divisible by 8 (0, 8, 16...).

CPUs are designed to read "aligned" memory efficiently. For example, if you read a 4-byte integer starting at address 0x03 (which is unaligned), the CPU has to do two memory reads — one for the first byte and another for the other three bytes — and then combine them. But if the integer starts at address 0x04 (which is aligned), the CPU can read all four bytes at once.

Aligned memory is also needed for vectorized CPU operations (SIMD), where one

processor instruction handles a group of values at once instead of just one.

The compiler knows the size and alignment for each type, so we can use the `Layout` constructor or helper functions to create a valid layout:

```
use std::alloc::Layout;

// 64-bit integer.
let i64_layout = Layout::new::<i64>();
println!("{:?}", i64_layout);

// Ten 32-bit integers.
let array_layout = Layout::array::<i32>(10).unwrap();
println!("{:?}", array_layout);

// Custom structure.
struct Cat {
    name: String,
    is_grumpy: bool,
}

let cat_layout = Layout::new::<Cat>();
println!("{:?}", cat_layout);

// Layout from a value.
let fluffy = Cat {
    name: String::from("Fluffy"),
    is_grumpy: true,
};

let fluffy_layout = Layout::for_value(&fluffy);
println!("{:?}", fluffy_layout);
```

Run ▶ Edit

> Don't be surprised that a `Cat` takes up 32 bytes. In Rust, the `String` type can grow, so it stores a data pointer, a length, and a capacity (3 × 8 = 24 bytes). There's also 1 byte for the boolean and 7 bytes of padding (because of 8-byte alignment), making a total of 32 bytes.

System allocator

`System` is the default memory allocator provided by the operating system. The exact implementation depends on the platform. It implements the `GlobalAlloc` trait and is used as the global allocator by default, but the documentation does not guarantee this (remember the "unspecified" note?). If you want to explicitly set `System` as the global allocator, you can use the `#[global_allocator]` attribute:

```rust
use std::alloc::System;

#[global_allocator]
static GLOBAL: System = System;

fn main() {
    // ...
}
```

You can also set a custom allocator as global, like `jemalloc` in this example:

```rust
use jemallocator::Jemalloc;

#[global_allocator]
static GLOBAL: Jemalloc = Jemalloc;

fn main() {}
```

Allocation helpers

To use the global allocator directly, call the `alloc` and `dealloc` functions:

```rust
use std::alloc::{alloc, dealloc, Layout};

unsafe {
    let layout = Layout::new::<u16>();
    let ptr = alloc(layout); // no OOM check for now
    dealloc(ptr, layout);
}
```

Run ▶ Edit

In practice, people rarely use `alloc` or `dealloc` directly. Instead, they work with types like `Box`, `String` or `Vec` that handle allocation for them:

```rust
let num = Box::new(42); // allocates
println!("{:?}", num);

let mut vec = Vec::new();
vec.push(1); // allocates
vec.push(2);
println!("{:?}", vec);

// num and vec automatically deallocate
// when they go out of scope.
```

Run ▶ Edit

Error handling

The `System` allocator doesn't abort if it can't allocate memory; instead, it returns `null` (which is exactly what `GlobalAlloc` recommends):

```
use std::alloc::{alloc, dealloc, handle_alloc_error, Layout};

unsafe {
    // Attempt to allocate a ton of memory.
    let layout = Layout::array::<u8>(usize::MAX / 2).unwrap();
    let ptr = alloc(layout);

    if ptr.is_null() {
        println!("Out of memory!");
        // Uncomment to abort.
        // handle_alloc_error(layout);
    } else {
        println!("Allocation succeeded.");
        dealloc(ptr, layout);
    }
}
```

Run ▶ Edit

The documentation recommends using the `handle_alloc_error` function to signal out-of-memory errors. It immediately aborts the process, or panics if the binary isn't linked to the standard library.

Unlike the low-level `alloc` function, types like `Box` or `Vec` call `handle_alloc_error` if allocation fails, so the program usually aborts if it runs out of memory:

```
let v: Vec<u8> = Vec::with_capacity(usize::MAX/2);
println!("{}", v.len());
```

Run ▶ Edit

Further reading

Allocator API • Memory allocation APIs

Zig

Memory management in Zig is explicit. There is no default global allocator, and any function that needs to allocate memory accepts an allocator as a separate

parameter. This makes the code a bit more verbose, but it matches Zig's goal of giving programmers as much control and transparency as possible.

Allocator interface

An allocator in Zig is a `std.mem.Allocator` struct with an opaque self-pointer and a method table with four methods:

```
const Allocator = @This();

ptr: *anyopaque,
vtable: *const VTable,

pub const VTable = struct {
    /// Return a pointer to `len` bytes with specified `alignment`,
    /// or return `null` indicating the allocation failed.
    alloc: *const fn (*anyopaque, len: usize, alignment: Alignment,
                      ret_addr: usize) ?[*]u8,

    /// Attempt to expand or shrink memory in place.
    resize: *const fn (*anyopaque, memory: []u8, alignment: Alignment,
                       new_len: usize, ret_addr: usize) bool,

    /// Attempt to expand or shrink memory, allowing relocation.
    remap: *const fn (*anyopaque, memory: []u8, alignment: Alignment,
                      new_len: usize, ret_addr: usize) ?[*]u8,

    /// Free and invalidate a region of memory.
    free: *const fn (*anyopaque, memory: []u8, alignment: Alignment,
                     ret_addr: usize) void,
};
```

Unlike Rust's allocator methods, which take a raw pointer and a size as arguments, Zig's allocator methods take a slice of bytes (`[]u8`) — a type that combines both a pointer and a length.

Another interesting difference is the optional `ret_addr` parameter, which is the first return address in the allocation call stack. Some allocators, like the `DebugAllocator`, use it to keep track of which function requested memory. This helps with debugging issues related to memory allocation.

Just like in Rust, allocator methods don't return errors. Instead, `alloc` and `remap` return `null` if they fail.

Allocation helpers

Zig also provides type-safe wrappers that you can use instead of calling the allocator methods directly:

```
// Allocate / deallocate a single object.
pub fn create(a: Allocator, comptime T: type) Error!*T
pub fn destroy(self: Allocator, ptr: anytype) void

// Allocate / deallocate multiple objects.
pub fn alloc(self: Allocator, comptime T: type, n: usize) Error![]T
pub fn free(self: Allocator, memory: anytype) void
```

Example:

```
const allocator = std.heap.page_allocator;

// Create and destroy a single integer.
const num = try allocator.create(i32);
num.* = 42;
allocator.destroy(num);

// Allocate and free a slice of 10 bytes.
const slice = try allocator.alloc(u8, 100);
@memset(slice, 'A');
allocator.free(slice);
```

Run ▶ Edit

Unlike the allocator methods, these allocation functions return an error if they fail.

If a function or method allocates memory, it expects the developer to provide an allocator instance:

```
const allocator = std.heap.page_allocator;

var list: std.ArrayList(u8) = .empty;
defer list.deinit(allocator);

try list.append(allocator, 'z');
try list.append(allocator, 'i');
try list.append(allocator, 'g');
```

Run ▶ Edit

Standard allocators

Zig's standard library includes several built-in allocators in the `std.heap` namespace.

`page_allocator` asks the operating system for entire pages of memory, each allocation is a syscall:

```
const allocator = std.heap.page_allocator;
const memory = try allocator.alloc(u8, 100);
allocator.free(memory);
```

Run ▶ Edit

`FixedBufferAllocator` allocates memory into a fixed buffer and doesn't make any heap allocations:

```
var buffer: [1000]u8 = undefined;
var fba: std.heap.FixedBufferAllocator = .init(&buffer);
const allocator = fba.allocator();

const memory = try allocator.alloc(u8, 100);
allocator.free(memory);
```

Run ▶ Edit

`ArenaAllocator` wraps a child allocator and allows you to allocate many times and only free once:

```
var arena: std.heap.ArenaAllocator = .init(std.heap.page_allocator);
defer arena.deinit();

const allocator = arena.allocator();

const mem1 = try allocator.alloc(u8, 100);
const mem2 = try allocator.alloc(u8, 100);
allocator.free(mem1); // not needed
allocator.free(mem2); // not needed
```

Run ▶ Edit

The `arena.deinit()` call frees all memory. Individual `allocator.free()` calls are no-ops.

`DebugAllocator` (aka `GeneralPurposeAllocator`) is a safe allocator that can prevent double-free, use-after-free and can detect leaks:

```
var gpa: std.heap.DebugAllocator(.{}) = .init;
const allocator = gpa.allocator();

const memory = try allocator.alloc(u8, 100);
allocator.free(memory);
allocator.free(memory); // aborts
```

`SmpAllocator` is a general-purpose thread-safe allocator designed for maximum performance on multithreaded machines:

```
const allocator = std.heap.smp_allocator;
const memory = try allocator.alloc(u8, 100);
allocator.free(memory);
```

Run ▶ Edit

`c_allocator` is a wrapper around the libc allocator:

```
const allocator = std.heap.c_allocator; // requires linking libc
const memory = try allocator.alloc(u8, 100);
allocator.free(memory);
```

Error handling

Zig doesn't panic or abort when it can't allocate memory. An allocation failure is just a regular error that you're expected to handle:

```
const allocator = std.heap.page_allocator;
const n = std.math.maxInt(i64);
const memory = allocator.alloc(u8, n) catch |err| {
    if (err == error.OutOfMemory) {
        print("Out of memory!\n", .{});
    }
    return err;
};
defer allocator.free(memory);
```

Run ▶ Edit

Further reading

Allocators • std.mem.Allocator • std.heap

Odin

Odin supports explicit allocators, but, unlike Zig, it's not the only option. In Odin, every scope has an implicit `context` variable that provides a default allocator:

```
Context :: struct {
    allocator:          Allocator,
```

```
    temp_allocator:     Allocator,
    // ...
}

// Returns the default `context` for each scope
@(require_results)
default_context :: proc "contextless" () -> Context {
    c: Context
    __init_context(&c)
    return c
}
```

If you don't pass an allocator to a function, it uses the one currently set in the context.

Allocator interface

An allocator in Odin is a `runtime.Allocator` struct with an opaque self-pointer and a single function pointer:

```
Allocator_Mode :: enum byte {
    Alloc,
    Free,
    Resize,
    // ...
}

Allocator_Error :: enum byte {
    None                 = 0,
    Out_Of_Memory        = 1,
    // ...
}

Allocator_Proc :: #type proc(
    allocator_data: rawptr,
    mode: Allocator_Mode,
    size, alignment: int,
    old_memory: rawptr,
    old_size: int,
    location: Source_Code_Location = #caller_location,
) -> ([]byte, Allocator_Error)

Allocator :: struct {
    procedure: Allocator_Proc,
    data:      rawptr,
}
```

Unlike other languages, Odin's allocator uses a single procedure for all allocation tasks. The specific action — like allocating, resizing, or freeing memory — is decided by the `mode` parameter.

The allocation procedure returns the allocated memory (for `.Alloc` and `.Resize`

operations) and an error (`.None` on success).

Allocation helpers

Odin provides low-level wrapper functions in the `core:mem` package that call the allocator procedure using a specific mode:

```
alloc :: proc(
    size: int,
    alignment: int = DEFAULT_ALIGNMENT,
    allocator := context.allocator,
    loc := #caller_location,
) -> (rawptr, runtime.Allocator_Error)

free :: proc(
    ptr: rawptr,
    allocator := context.allocator,
    loc := #caller_location,
) -> runtime.Allocator_Error

// and others
```

There are also type-safe builtins like `new` / `free` (for a single object) and `make` / `delete` (for multiple objects) that you can use instead of the low-level interface:

```
num := new(int)
defer free(num)

slice := make([]int, 100)
defer delete(slice)
```

Run ▶ Edit

By default, all builtins use the context allocator, but you can pass a custom allocator as an optional parameter:

```
ptr := new(int, allocator=context.allocator)
defer free(ptr, allocator=context.allocator)

slice := make([]int, 10, allocator=context.allocator)
defer delete(slice, allocator=context.allocator)
```

Run ▶ Edit

To use a different allocator for a specific block of code, you can reassign it in the

context:

```
alloc := custom_allocator()
context.allocator = alloc

// Uses the custom allocator.
ptr := new(int)
defer free(ptr)
```

Temp allocator

Odin's `context` provides two different allocators:

> `context.allocator` is for general-purpose allocations. It uses the operating system's heap allocator.
>
> `context.temp_allocator` is for short-lived allocations. It uses a scratch allocator (a kind of growing arena).

```
// Temporary allocation (no manual free required).
temp_mem, _ := mem.alloc(100, allocator=context.temp_allocator)

// Persistent allocation (requires manual free).
perm_mem, _ := mem.alloc(100, allocator=context.allocator)
defer mem.free(perm_mem, context.allocator)

// Clear the entire scratchpad at the end of the work cycle.
free_all(context.temp_allocator)
```

Run ▶ Edit

When using the temp allocator, you only need a single `free_all` call to clear all the allocated memory.

Standard allocators

Odin's standard library includes several allocators, found in the `base:runtime` and `core:mem` packages.

The `heap_allocator` procedure returns a general-purpose allocator:

```
allocator := runtime.heap_allocator()
memory, err := mem.alloc(100, allocator=allocator)
mem.free(memory, allocator=allocator)
```

Run ▶ Edit

`Arena` uses a single backing buffer for allocations, allowing you to allocate many times and only free once:

```
arena: mem.Arena
buffer := make([]byte, 1024, runtime.heap_allocator())
mem.arena_init(&arena, buffer)
defer mem.arena_free_all(&arena)

allocator := mem.arena_allocator(&arena)
m1, err1 := mem.alloc(100, allocator=allocator)
m2, err2 := mem.alloc(100, allocator=allocator)
```

Run ▶ Edit

`Tracking_Allocator` detects leaks and invalid memory access, similar to `DebugAllocator` in Zig:

```
track: mem.Tracking_Allocator
mem.tracking_allocator_init(&track, runtime.default_allocator())
defer mem.tracking_allocator_destroy(&track)

allocator := mem.tracking_allocator(&track)
memory, err := mem.alloc(100, allocator=allocator)
free(memory, allocator=allocator)
free(memory, allocator=allocator) // aborts
```

Run ▶ Edit

There are also others, such as `Stack` or `Buddy_Allocator`.

Error handling

Like Zig, Odin doesn't panic or abort when it can't allocate memory. Instead, it returns an error code as the second return value:

```
data, err := mem.alloc(1 << 62)
if err != .None {
    fmt.println("Allocation failed:", err)
    return
}
defer mem.free(data)
```

Run ▶ Edit

Further reading

Allocators • base:runtime • core:mem

C3

Like Zig and Odin, C3 supports explicit allocators. Like Odin, C3 provides two default allocators: heap and temp.

Allocator interface

An allocator in C3 is a `core::mem::allocator::Allocator` interface with an additional option of zeroing or not zeroing the allocated memory:

```
enum AllocInitType
{
    NO_ZERO,
    ZERO
}

interface Allocator
{
    <*
     Acquire memory from the allocator, with the given
     alignment and initialization type.
    *>
    fn void*? acquire(usz size, AllocInitType init_type, usz alignment = 0);

    <*
     Resize acquired memory from the allocator,
     with the given new size and alignment.
    *>
    fn void*? resize(void* ptr, usz new_size, usz alignment = 0);

    <*
     Release memory acquired using `acquire` or `resize`.
    *>
    fn void release(void* ptr, bool aligned);
}
```

Unlike Zig and Odin, the `resize` and `release` methods don't take the (old) size as a parameter — neither directly like Odin nor through a slice like Zig. This makes it a bit harder to create custom allocators because the allocator has to keep track of the size along with the allocated memory. On the other hand, this approach makes C interop easier (if you use the default C3 allocator): data allocated in C can be freed in C3 without needing to pass the size parameter from the C code.

Like in Odin, allocator methods return an error if they fail.

Allocation helpers

C3 provides low-level wrapper macros in the `core::mem::allocator` module that call allocator methods:

```
macro void* malloc(Allocator allocator, usz size)
macro void*? malloc_try(Allocator allocator, usz size)

macro void* realloc(Allocator allocator, void* ptr, usz new_size)
macro void*? realloc_try(Allocator allocator, void* ptr, usz new_size)

macro void free(Allocator allocator, void* ptr)

// and others
```

These either return an error (the `_try`-suffix macros) or abort if they fail.

Example:

```
// `mem` is the global allocator instance.
int* ptr = allocator::malloc(mem, int.sizeof);
defer allocator::free(mem, ptr);
```

Run ▶ Edit

There are also functions and macros with similar names in the `core::mem` module that use the global `allocator::mem` allocator instance:

```
// Call the core::mem::allocator macros directly.
fn void* malloc(usz size)
fn void free(void* ptr)

// Accept a type instead of a size.
macro new($Type, #init = ...)
macro alloc($Type)

// Allocate multiple objects.
macro new_array($Type, usz elements)
macro alloc_array($Type, usz elements)

// and others
```

Example:

```
// `malloc` and `free` are builtins,
// so they don't require the namespace.
int* num = malloc(int.sizeof);
defer free(num);

// `new_array` requires the namespace.
int[] slice = mem::new_array(int, 100);
defer free(slice);
```

Run ▶ Edit

If a function or method allocates memory, it often expects the developer to provide an allocator instance:

```
List{int} list;
list.init(mem); // use the heap allocator
defer list.free();

list.push(11);
list.push(22);
list.push(33);
```

Run ▶ Edit

Temp allocator

C3 provides two thread-local allocator instances:

- `allocator::mem` is for general-purpose allocations. It uses a operating system's heap allocator (typically a libc wrapper).
- `allocator::tmem` is for short-lived allocations. It uses an arena allocator.

There are functions and macros in the `core::mem` module that use the `allocator::tmem` temporary allocator:

```
// Calls the core::mem::allocator macro directly.
fn void* tmalloc(usz size, usz alignment = 0)

// Accept a type instead of a size.
macro tnew($Type, #init = ...)
macro talloc($Type)

// Allocate multiple objects.
macro talloc_array($Type, usz elements)
```

To `@pool` macro releases all temporary allocations when leaving the scope:

```
@pool()
{
    int* p1 = tmalloc(int.sizeof);
    int* p2 = tmalloc(int.sizeof);
    int* p3 = tmalloc(int.sizeof);
    // no manual free required
};  // p1, p2, p3 are freed here
```

Run ▶ Edit

Some types, like `List` or `DString`, use the temp allocator by default if they are not initialized:

```
@pool()
{
    List{int} list;
    list.push(11);  // implicitly initialize with the temp allocator
    list.push(22);

    DString str;
    str.appendf("Hello %s", "World");  // same
};
```

Run ▶ Edit

Standard allocators

C3's standard library includes several built-in allocators, found in the `core::mem::allocator` module.

`LibcAllocator` is a wrapper around libc's malloc/free:

```
LibcAllocator libc;
char* memory = allocator::malloc(&libc, 100*char.sizeof);
allocator::free(&libc, memory);
```

Run ▶ Edit

`ArenaAllocator` uses a single backing buffer for allocations, allowing you to allocate many times and only free once:

```
char[1024] buf;
ArenaAllocator* arena = allocator::wrap(&buf);
defer arena.clear();

char* m1 = allocator::malloc(arena, 100*char.sizeof);
char* m2 = allocator::malloc(arena, 100*char.sizeof);
```

Run ▶ Edit

`TrackingAllocator` detects leaks and invalid memory access:

```
TrackingAllocator track;
track.init(mem);
defer track.clear();

char* memory = allocator::malloc(&track, 100*char.sizeof);
allocator::free(&track, memory);
allocator::free(&track, memory); // aborts
```

Run ▶ Edit

There are also others, such as `BackedArenaAllocator` or `OnStackAllocator`.

Error handling

Like Zig and Odin, C3 can return an error in case of allocation failure:

```
void*? data = allocator::malloc_try(mem, 1uLL << 62);
if (catch err = data) {
    io::printfn("Allocation failed: %s", err);
    return;
};
defer mem::free(data);
```

Run ▶ Edit

C3 can also abort in case of allocation failure:

```
void* data = allocator::malloc(mem, 1uLL << 62);
// void* data = malloc(1uLL << 62); // same thing
defer free(data);
```

Run ▶ Edit

Since the functions and macros in the `core::mem` module use `allocator::malloc` instead of `allocator::malloc_try`, it looks like aborting on failure is the preferred approach.

Further reading

Memory Handling • core::mem::alocator • core::mem

Hare

Unlike other languages, Hare doesn't support explicit allocators. The standard library has multiple allocator implementations, but only one of them is used at

runtime.

Global allocator

Hare's compiler expects the runtime to provide `malloc` and `free` implementations:

```
fn malloc(n: size) nullable *opaque;
@symbol("rt.free") fn free_(_p: nullable *opaque) void;
```

The programmer isn't supposed to access them directly (although it's possible by importing `rt` and calling `rt::malloc` or `rt::free`). Instead, Hare uses them to provide higher-level allocation helpers.

Allocation helpers

Hare offers two high-level allocation helpers that use the global allocator internally: `alloc` and `free`.

`alloc` can allocate individual objects. It takes a value, not a type:

```
let n: *int = alloc(42)!;
defer free(n);

let s: *str = alloc("hello world")!;
defer free(s);

// coords is defined as struct { x: int, y: int }
let p: *coords = alloc(coords{x=3, y=5})!;
defer free(p);
```

Run ▶ Edit

`alloc` can also allocate slices if you provide a second parameter (the number of items):

```
// Allocate a slice of 100 integers.
let nums: []int = alloc([0...], 100)!;
defer free(nums);
```

Run ▶ Edit

`free` works correctly with both pointers to single objects (like `*int`) and slices (like `[]int`).

Standard allocators

Hare's standard library has three built-in memory allocators:

> The default allocator is based on the algorithm from the Verified sequential malloc/free paper.
>
> The libc allocator uses the operating system's malloc and free functions from libc.
>
> The debug allocator uses a simple mmap-based method for memory allocation.

The allocator that's actually used is selected at compile time.

Error handling

Like other languages, Hare returns an error in case of allocation failure:

```
match (alloc([0...], 1 << 62)) {
case let nums: []int =>
    defer free(nums);
case nomem =>
    fmt::println("Out of memory")!;
};
```

Run ▶ Edit

You can abort on error with `!`:

```
let nums: []int = alloc([0...], 1 << 62)!;
defer free(nums);
```

Run ▶ Edit

Or propagate the error with `?`:

```
let nums: []int = alloc([0...], 1 << 62)?;
defer free(nums);
```

Further reading

Dynamic memory allocation • malloc.ha

C

Many C programs use the standard libc allocator, or at most, let you swap it out for

another one using macros:

```
#define LIB_MALLOC malloc
#define LIB_FREE free
```

Or using a simple setter:

```
static void *(*_lib_malloc)(size_t);
static void (*_lib_free)(void*);

void lib_set_allocator(void *(*malloc)(size_t), void (*free)(void*)) {
    _lib_malloc = malloc;
    _lib_free = free;
}
```

While this might work for switching the libc allocator to jemalloc or mimalloc, it's not very flexible. For example, trying to implement an arena allocator with this kind of API is almost impossible.

Now that we've seen the modern allocator design in Zig, Odin, and C3 — let's try building something similar in C. There are a lot of small choices to make, and I'm going with what I personally prefer. I'm not saying this is the only way to design an allocator — it's just one way out of many.

Allocator interface

Our allocator should return an error instead of `NULL` if it fails, so we'll need an error enum:

```
// Allocation errors.
typedef enum {
    Error_None = 0,
    Error_OutOfMemory,
    Error_SizeOverflow,
} Error;
```

The allocation function needs to return either a tagged union (value | error) or a tuple (value, error). Since C doesn't have these built in, let's use a custom tuple type:

```
// Allocation result.
typedef struct {
    void* ptr;
    Error err;
```

```
} AllocResult;
```

The next step is the allocator interface. I think Odin's approach of using a single function makes the implementation more complicated than it needs to be, so let's create separate methods like Zig does:

```
// Allocator interface.
struct _Allocator {
    AllocResult (*alloc)(void* self, size_t size, size_t align);
    AllocResult (*realloc)(void* self, void* ptr, size_t oldSize,
                           size_t newSize, size_t align);
    void (*free)(void* self, void* ptr, size_t size, size_t align);
};

typedef struct {
    const struct _Allocator* m;
    void* self;
} Allocator;
```

> This approach to interface design is explained in detail in a separate post: Interfaces in C.

Zig uses byte slices (`[]u8`) instead of raw memory pointers. We could make our own byte slice type, but I don't see any real advantage to doing that in C — it would just mean more type casting. So let's keep it simple and stick with `void*` like our ancestors did.

Allocation helpers

Now let's create generic `Alloc` and `Free` wrappers:

```
// Allocates an item of type T.
// `AllocResult Alloc[T](Allocator a, T)`
#define Alloc(a, T) \
    ((a).m->alloc((a).self, sizeof(T), alignof(T)))

// Frees an item allocated with Alloc.
// Only accepts typed pointers, not void*.
// `void Free[T](Allocator a, T* ptr)`
#define Free(a, ptr) \
    ((a).m->free((a).self, (ptr), sizeof(*(ptr)), alignof(typeof(*(ptr)))))
```

I'm taking `typeof` for granted here to keep things simple. A more robust implementation should properly check if it is available or pass the type to `Free` directly.

We can even create a separate pair of helpers for collections:

```
// Helper to prevent integer overflow during N-item allocation.
static inline size_t calcSize(size_t size, size_t count) {
    if (count > 0 && size > SIZE_MAX / count) {
        return 0;
    }
    return size * count;
}

// Allocates n items of type T.
// `AllocResult AllocN[T](Allocator a, T, size_t n)`
#define AllocN(a, T, n) \
    ((a).m->alloc((a).self, calcSize(sizeof(T), (n)), alignof(T)))

// Frees n items allocated with AllocN.
// Only accepts typed pointers, not void*.
// `void FreeN[T](Allocator a, T* ptr, size_t n)`
#define FreeN(a, ptr, n)                \
    ((a).m->free(                       \
        (a).self, (ptr),                \
        calcSize(sizeof(*(ptr)), (n)), \
        alignof(typeof(*(ptr)))))
```

We could use some `__VA_ARGS__` macro tricks to make `Alloc` and `Free` work for both a single object and a collection. But let's not do that — I prefer to avoid heavy-magic macros in this post.

Libc allocator

As for the custom allocators, let's start with a libc wrapper. It's not particularly interesting, since it ignores most of the parameters, but still:

```
// The libc allocator wrapper.
// Ignores alignment and treats zero-size allocations as errors.
// Doesn't support reallocation to keep things simple.
AllocResult Libc_Alloc(void* self, size_t size, size_t align) {
    (void)self;
    (void)align;

    if (size == 0) return (AllocResult){NULL, Error_SizeOverflow};
    void* ptr = malloc(size);
    if (!ptr) return (AllocResult){NULL, Error_OutOfMemory};
    return (AllocResult){ptr, Error_None};
}

void Libc_Free(void* self, void* ptr, size_t size, size_t align) {
    (void)self;
    (void)size;
    (void)align;
    free(ptr);
}

Allocator LibcAllocator(void) {
    static const struct _Allocator mtab = {
```

```
        .alloc = Libc_Alloc,
        .free = Libc_Free,
    };
    return (Allocator){.m = &mtab, .self = NULL};
}
```

Edit

Usage example:

```
int main(void) {
    Allocator allocator = LibcAllocator();

    {
        // Allocate a single integer.
        AllocResult res = Alloc(allocator, int64_t);
        if (res.err != Error_None) {
            printf("Error: %d\n", res.err);
            return 1;
        }

        int64_t* x = res.ptr;
        *x = 42;

        Free(allocator, x);
    }

    {
        // Allocate an array of integers.
        size_t n = 100;
        AllocResult res = AllocN(allocator, int64_t, n);
        if (res.err != Error_None) {
            printf("Error: %d\n", res.err);
            return 1;
        }

        int64_t* arr = res.ptr;
        for (size_t i = 0; i < n; i++) {
            arr[i] = i + 1;
        }

        FreeN(allocator, arr, n);
    }
}
```

Run ▶ Edit

Arena allocator

Now let's use that `self` field to implement an arena allocator backed by a fixed-size buffer:

```c
// A simple arena allocator.
// Doesn't support reallocation.
typedef struct {
    uint8_t* buf;
    size_t cap;
    size_t offset;
} Arena;

Arena NewArena(uint8_t* buf, size_t cap) {
    return (Arena){.buf = buf, .cap = cap, .offset = 0};
}

static AllocResult Arena_Alloc(void* self, size_t size, size_t align) {
    Arena* arena = (Arena*)self;

    // 1. Calculate the alignment padding.
    if (size == 0) return (AllocResult){NULL, Error_SizeOverflow};
    uintptr_t currentPtr = (uintptr_t)arena->buf + arena->offset;
    uintptr_t alignedPtr = (currentPtr + (align - 1)) & ~(align - 1);
    size_t newOffset = (alignedPtr - (uintptr_t)arena->buf) + size;

    // 2. Check for errors.
    if (newOffset < arena->offset) {
        return (AllocResult){NULL, Error_SizeOverflow};
    }
    if (newOffset > arena->cap) {
        return (AllocResult){NULL, Error_OutOfMemory};
    }

    // 3. Commit the allocation.
    arena->offset = newOffset;
    return (AllocResult){(void*)alignedPtr, Error_None};
}

static void Arena_Free(void* self, void* ptr, size_t size, size_t align) {
    // Individual deallocations are no-ops.
    (void)self;
    (void)ptr;
    (void)size;
    (void)align;
}

static void Arena_Reset(Arena* arena) {
    arena->offset = 0;
}

Allocator Arena_Allocator(Arena* arena) {
    static const struct _Allocator mtab = {
        .alloc = Arena_Alloc,
        .free = Arena_Free,
    };
    return (Allocator){.m = &mtab, .self = arena};
}
```

Edit

Usage example:

```c
int main(void) {
    uint8_t buf[1024];
    Arena arena = NewArena(buf, sizeof(buf));
    Allocator allocator = Arena_Allocator(&arena);

    {
        // Allocate a single integer.
        AllocResult res = Alloc(allocator, int64_t);
        if (res.err != Error_None) {
            printf("Error: %d\n", res.err);
            return 1;
        }

        int64_t* x = res.ptr;
        *x = 42;

        // No need for Free.
    }

    {
        // Allocate an array of integers.
        size_t n = 100;
        AllocResult res = AllocN(allocator, int64_t, n);
        if (res.err != Error_None) {
            printf("Error: %d\n", res.err);
            return 1;
        }

        int64_t* arr = res.ptr;
        for (size_t i = 0; i < n; i++) {
            arr[i] = i + 1;
        }

        // No need for FreeN.
    }

    Arena_Reset(&arena);
}
```

Run ▶ Edit

Nice!

Error handling

As shown in the examples above, the allocation method returns an error if something goes wrong. While checking for errors might not be as convenient as it is in Zig or Odin, it's still pretty straightforward:

```c
int main(void) {
    Allocator allocator = LibcAllocator();

    size_t n = SIZE_MAX;
```

```
    AllocResult res = AllocN(allocator, int64_t, n);
    if (res.err != Error_None) {
        printf("Allocation failed: %d\n", res.err);
        return 1;
    }

    FreeN(allocator, res.ptr, n);
}
```

Run ▶ Edit

source

Final thoughts

Here's an informal table comparing allocation APIs in the languages we've discussed:

	Single object	Collection
Rust	`Box::new(42)`	`vec![0; 100]`
Zig	`a.create(i32)`	`a.alloc(i32, 100)`
Odin	`new(int)` `new(int, a)`	`make([]int, 100)` `make([]int, 100, a)`
C3	`mem::new(int)`	`mem::new_array(int, 100)`
Hare	`alloc(42)`	`alloc([0...], 100)`
C	`Alloc(a, int)`	`AllocN(a, int, 100)`

In Zig, you always have to specify the allocator. In Odin, passing an allocator is optional. In C3, some functions require you to pass an allocator, while others just use the global one. In Hare, there's a single global allocator.

As we've seen, there's nothing magical about the allocators used in modern languages. While they're definitely more ergonomic and safe than C, there's nothing stopping us from using the same techniques in plain C.

12 Feb, 2026 can-c